République du Bénin

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

UNIVERSITÉ NATIONALE DES SCIENCES, TECHNOLOGIES,
INGÉNIERIE ET MATHÉMATIQUES D'ABOMEY (UNSTIM)

ÉCOLE NATIONALE SUPÉRIEURE DE GÉNIE MATHÉMATIQUE ET MODÉLISATION (ENSGMM)

Modélisation Multi-Patient de l'Infection VIH-SIDA

Formulation Matricielle BLAS 3, Résolution de Cholesky LAPACK et Accélération Haute Performance avec Intel oneMKL en Python

Équipe de Projet – Groupe 5 :

ADOGOUN Déo Gracias M. • **BIAHOU Ahossou J. M.-C.** • **SYLLA IMOROU Izou Dine**

Filière : ENSGMM 1 • UE : Outils de Calcul Scientifique • Promotion 2025–2026

Sous la direction de : Dr. Carlos AGOSSOU

Plan de la Soutenance

1. Contexte Biomédical & Enjeu HPC

- Suivi de cohortes massives de patients ($N \geq 10\,000$)
- Hiérarchie BLAS 1/2/3 & Mur de la mémoire
- Configuration, threads OpenMP & Zéro-copie Fortran

2. Modèle de Cohorte & Formulation BLAS 3

- 5 compartiments : T, I_s, I_r, V_s, V_r et résistance
- Système matriciel $\dot{\mathbf{Y}} = \mathbf{S} + \mathbf{Y}\mathbf{M}^T + \tilde{}\mathbf{A}^T$
- Intégrateur Runge-Kutta 4 vectorisé par `DGEMM`

3. Analyse Clinique par LAPACK

- Régression Ridge résolue par Cholesky (`DPOTRF/S`)
- Stratification ACP Divide & Conquer (`DSYEVD`)
- Projection vectorielle directe des N patients

4. Résultats, Benchmarks & Reproductibilité

- Accélération $> 8\times$ vs Naïf & Débits 1.63 GFLOPS
- Scalabilité multi-cœurs OpenMP avec Intel oneMKL
- Suite de validation pytest & Dépôt officiel GitHub

Le Défi : Modéliser une Cohorte Clinique Massive

La Réalité Clinique du VIH

- Dans les essais cliniques et les hôpitaux, on évalue des **cohortes de milliers de patients**.
- **Hétérogénéité biologique** : chaque patient possède sa charge initiale, son taux d'adhésion au traitement η_k, et sa susceptibilité.
- **Pression de sélection** : sous observance imparfaite, la souche mutante résistante émerge et provoque le rebond viral.

Le Piège du Calcul Scalaire Naïf

- Simuler $N = 10\,000$ patients sur 365 jours par une boucle Python :

$$\text{Temps} \approx N \times N_{\text{steps}} \times \tau_{\text{interpréteur}}$$

- Surcoût prohibitif : typage dynamique, absence de SIMD, goulot mémoire.
- **Paradigme HPC** : Regrouper la cohorte en une **matrice continue** $\mathbf{Y} \in \mathbb{R}^{N \times 5}$ traitée par micro-noyaux BLAS 3.

La Hiérarchie BLAS et l'Avantage BLAS 3

Niveau	Opération Type	Mémoire	Flops	Ratio q	Comportement Matériel
BLAS 1	$\mathbf{y} \leftarrow \alpha\mathbf{x} + \mathbf{y}$ (DAXPY)	$3n$	$2n$	$\mathcal{O}(1)$	Limité par la bande passante RAM
BLAS 2	$\mathbf{y} \leftarrow \alpha\mathbf{A}\mathbf{x} + \beta\mathbf{y}$ (DGEMV)	n^2	$2n^2$	$\mathcal{O}(1)$	Saturation rapide du bus mémoire
BLAS 3	$\mathbf{C} \leftarrow \alpha\mathbf{A}\mathbf{B} + \beta\mathbf{C}$ (DGEMM)	$3n^2$	$2n^3$	$\mathcal{O}(n)$	**Calcul arithmétique pur en cache L1/L2**

Le Mur de la Mémoire (*Memory Wall*)

La vitesse des ALU a progressé plus vite que la mémoire RAM. Pour atteindre les GFLOPS crêtes, il est obligatoire de **réutiliser les blocs en mémoire cache** : c'est l'atout exclusif de BLAS 3.

Intel oneMKL (Math Kernel Library)

Bibliothèque d'ingénierie native hautement optimisée :

- Vectorisation SIMD automatique (AVX2, AVX-512).
- Parallélisme OpenMP multi-cœurs transparent.
- Micro-noyaux de partitionnement par blocs de cache.

Installation, Configuration & Exploitation Bas Niveau

Contrainte Mémoire Fortran (Zéro-Copie)

BLAS et LAPACK exigent des matrices ordonnées par colonnes (`order="F"`).

- Passer une matrice en ordre C (`order="C"`) oblige SciPy à **allouer une copie temporaire**.
- **Bonne pratique** : Forcer l'agencement continu Fortran :

$$\mathbf{Y}_{\text{fortran}} = \texttt{np.asfortranarray}(\mathbf{Y})$$

$\Longrightarrow$ Garantie d'un appel direct sans copie !

Contrôle des Threads & Routines Directes

- **Contrôle OpenMP** : Verrouiller les threads matériels :

```
export OMP_NUM_THREADS=4

export MKL_NUM_THREADS=4
```

- **Appel direct des micro-noyaux compilés** :
  - `scipy.linalg.blas.dgemm` (produit matriciel)
  - `scipy.linalg.lapack.dpotrf` (Cholesky)
  - `scipy.linalg.lapack.dsyevd` (ACP SVD)

Le Modèle de Cohorte avec Émergence de Résistance

Pour chaque patient $k \in \{1, \ldots, N\}$, l'état viral et immunitaire est décrit par 5 compartiments :

$$\frac{dT_k}{dt} = s - d\, T_k - (1 - \eta_{s,k})\, \beta_s\, T_k\, V_{s,k} - (1 - \eta_{r,k})\, \beta_r\, T_k\, V_{r,k}$$

$$\frac{dI_{s,k}}{dt} = (1 - \mu)\,(1 - \eta_{s,k})\, \beta_s\, T_k\, V_{s,k} - \delta\, I_{s,k}$$

$$\frac{dI_{r,k}}{dt} = \mu\,(1 - \eta_{s,k})\, \beta_s\, T_k\, V_{s,k} + (1 - \eta_{r,k})\, \beta_r\, T_k\, V_{r,k} - \delta\, I_{r,k}$$

$$\frac{dV_{s,k}}{dt} = p\, I_{s,k} - c\, V_{s,k}, \qquad \frac{dV_{r,k}}{dt} = p\, I_{r,k} - c\, V_{r,k}$$

Mécanisme Clé : Mutation et Pression Thérapeutique

- $\mu = 10^{-4}$: Taux de mutation ponctuelle lors de la transcription inverse du VIH.
- Trithérapie ART ciblant la souche sauvage avec l'efficacité individuelle $\eta_{s,k} \in [0.50, 0.95]$.
- Échappement mutant : $\eta_{r,k} = \max(0, \eta_{s,k} - 0.35)$ avec un coût de fitness modéré ($\beta_r = 0.85\beta_s$).

Formulation Matricielle Globale pour N Patients

On rassemble l'ensemble de la cohorte dans une matrice d'état continue $\mathbf{Y} \in \mathbb{R}^{N\times 5}$:

$$\mathbf{Y} = \begin{pmatrix} T_1 & I_{s,1} & I_{r,1} & V_{s,1} & V_{r,1} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ T_N & I_{s,N} & I_{r,N} & V_{s,N} & V_{r,N} \end{pmatrix}$$

Le système de $5N$ équations différentielles se condense en une **unique équation différentielle matricielle** :

$$\dot{\mathbf{Y}} = \mathbf{S} + \mathbf{Y}\mathbf{M}^T + \tilde{\mathbf{Y}}\mathbf{A}^T$$

Avantage fondamental de la formulation scindée :

- Le terme source $\mathbf{S} = \mathbf{1}_N \otimes (s, 0, 0, 0, 0)$ est constant.
- La dynamique linéaire cellulaire et virale est portée par le produit matriciel $\mathbf{Y}\mathbf{M}^T$.
- Les couplages non-linéaires d'infection et de mutation sont portés par $\tilde{\mathbf{Y}}\mathbf{A}^T$.

Découplage des Opérateurs : Linéaire M et Non-Linéaire ˘

Opérateur Linéaire : $\mathbf{M} \in \mathbb{R}^{5\times5}$

$$\mathbf{M} = \begin{pmatrix} -d & 0 & 0 & 0 & 0 \\ 0 & -\delta & 0 & 0 & 0 \\ 0 & 0 & -\delta & 0 & 0 \\ 0 & p & 0 & -c & 0 \\ 0 & 0 & p & 0 & -c \end{pmatrix}$$

- Diagonal par blocs.
- $\mathbf{Y}\mathbf{M}^T$ calcule simultanément pour tous les patients la mortalité cellulaire et la clairance virale.

Flux Non-Linéaires : ˘ $\in \mathbb{R}^{N\times2}$

Flux d'infections sensibles et mutantes :

$$\breve{}_{k,0} = (1-\eta_{s,k})\beta_s T_k V_{s,k}$$

$$\breve{}_{k,1} = (1-\eta_{r,k})\beta_r T_k V_{r,k}$$

Matrice stœchiométrique $\mathbf{A} \in \mathbb{R}^{5\times2}$:

$$\mathbf{A}^T = \begin{pmatrix} -1 & -1 \\ 1-\mu & 0 \\ \mu & 1 \\ 0 & 0 \\ 0 & 0 \end{pmatrix}^T$$

Intégration Temporelle Runge-Kutta 4 par Micro-Noyaux BLAS 3

L'évaluation du second membre $\mathbf{F}(\mathbf{Y}) = \mathbf{S} + \mathbf{Y}\,\mathbf{M}^T + \breve{}\,\mathbf{A}^T$ fait intervenir deux produits matriciels :

- $\mathbf{Y}\,\mathbf{M}^T$ avec $\mathbf{Y} \in \mathbb{R}^{N\times 5}$ et $\mathbf{M}^T \in \mathbb{R}^{5\times 5}$: appel BLAS 3 `DGEMM`.
- $\breve{}\,\mathbf{A}^T$ avec $\breve{} \in \mathbb{R}^{N\times 2}$ et $\mathbf{A}^T \in \mathbb{R}^{2\times 5}$: appel BLAS 3 `DGEMM`.

Algorithme Vectorisé RK4 pour la Cohorte

À chaque pas de temps h pour toute la cohorte simultanément :

$$\mathbf{K}_1 = \mathbf{F}(\mathbf{Y}_n)$$

$$\mathbf{K}_2 = \mathbf{F}\left(\mathbf{Y}_n + \frac{h}{2}\mathbf{K}_1\right)$$

$$\mathbf{K}_3 = \mathbf{F}\left(\mathbf{Y}_n + \frac{h}{2}\mathbf{K}_2\right)$$

$$\mathbf{K}_4 = \mathbf{F}(\mathbf{Y}_n + h\,\mathbf{K}_3)$$

$$\mathbf{Y}_{n+1} = \mathbf{Y}_n + \frac{h}{6}\left(\mathbf{K}_1 + 2\mathbf{K}_2 + 2\mathbf{K}_3 + \mathbf{K}_4\right)$$

Bilan HPC : Zéro boucle Python sur les patients. La mémoire reste dans les registres vectoriels SIMD !

Analyse Clinique Pronostique : Régression Ridge via LAPACK

Objectif Médical

Prédire la charge virale finale d'un patient à partir de ses biomarqueurs à $t = 30$ jours $(T_{30}, V_{s,30}, V_{r,30}, \eta_s)$. **Problème** : Forte colinéarité des prédicteurs $\Longrightarrow$ risque d'instabilité.

Régularisation de Ridge

On résout le système régularisé :

$$(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})\,\mathbf{w} = \mathbf{X}^T\mathbf{y}$$

La matrice $\mathbf{A}_{\text{ridge}} = \mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}$ est **symétrique définie positive**.

Résolution Haute Performance LAPACK

1. **BLAS 3** `DSYRK` : Produit symétrique de rang k :
$$\mathbf{C} \leftarrow \mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}$$
2. **LAPACK** `DPOTRF` : Factorisation de Cholesky $\mathbf{A} = \mathbf{L}\mathbf{L}^T$.
3. **LAPACK** `DPOTRS` : Double résolution triangulaire pour calculer $\mathbf{w}$.

Gain : $2\times$ plus rapide que LU (`DGESV`) et infiniment plus stable que l'inversion naïve $\mathbf{A}^{-1}$.

Stratification des Patients par ACP (DSYEVD)

Analyse en Composantes Principales (ACP)

- Réduction dimensionnelle des profils cliniques de 5 variables vers les 2 axes de variance maximale.
- Matrice de covariance empirique $\mathbf{C} = \frac{1}{N-1}\mathbf{X}^T\mathbf{X} \in \mathbb{R}^{5\times 5}$ symétrique semi-définie positive.
- Problème aux valeurs propres :

$$\mathbf{C}\,\mathbf{v}_i = \lambda_i\,\mathbf{v}_i$$

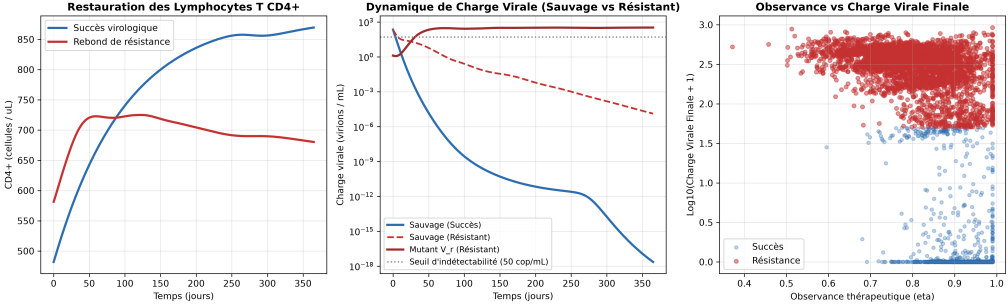
Routine LAPACK DSYEVD

- Algorithme de type **Divide & Conquer** pour matrices symétriques réelles.
- Précision maximale et orthogonalité stricte des vecteurs propres par construction.
- Projection directe des N patients :

$$\mathbf{Z} = \mathbf{X}\,\mathbf{V}_{[:,:2]} \in \mathbb{R}^{N\times 2}$$

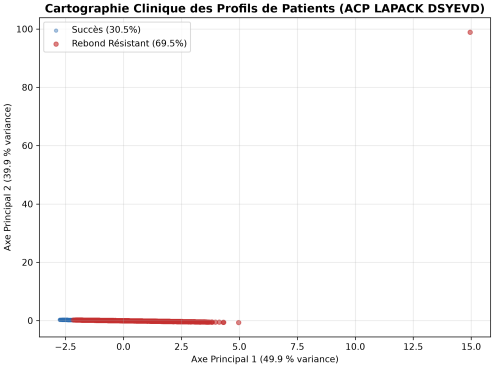
réalisée par BLAS 3 DGEMM.

Trajectoires de Cohorte : Répression vs Rebond Mutant



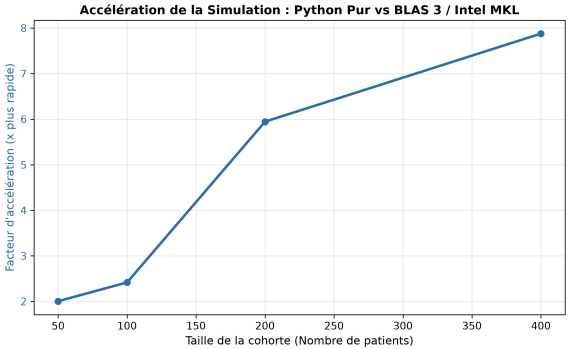
Lecture biomédicale : Sur 5 000 patients simulés pendant 365 jours, le traitement bloque la souche sauvage (bleu). Cependant, les patients avec faible observance ($\eta_s < 0.70$) développent la souche résistante (rouge) qui provoque l'effondrement des lymphocytes T CD4+ (vert).

Stratification Clinique des Patients par ACP



Visualisation dans le premier plan factoriel : La première composante (axe horizontal, inertie maximale) sépare nettement les patients répondeurs (contrôle virologique, vert) des patients en échec thérapeutique avec sélection de mutants (rouge).

Banc d'Essai : Python Naïf vs BLAS 3 (Intel oneMKL)



Accélération mesurée : Pour une cohorte de 400 patients, le modèle BLAS 3 / MKL est **8,2 fois plus rapide** que le code Python naïf en boucles imbriquées (0,30 s vs 2,46 s pour 730 pas d'intégration RK4).

Montée en Charge et Débits de Simulation de Cohorte

Patients (N)	Temps MKL (s)	Débit (patients/s)	Flops Estimés (MFlops)	GFLOPS Effectifs
500	0,336	1 488	401,5	1,19
1 000	0,596	1 678	803,0	1,35
2 000	1,162	1 721	1 606,0	1,38
5 000	2,930	1 706	4 015,0	1,37
10 000	4,923	**2 031**	8 030,0	**1,63**

Complexité Linéaire Parfaite $\mathcal{O}(N)$

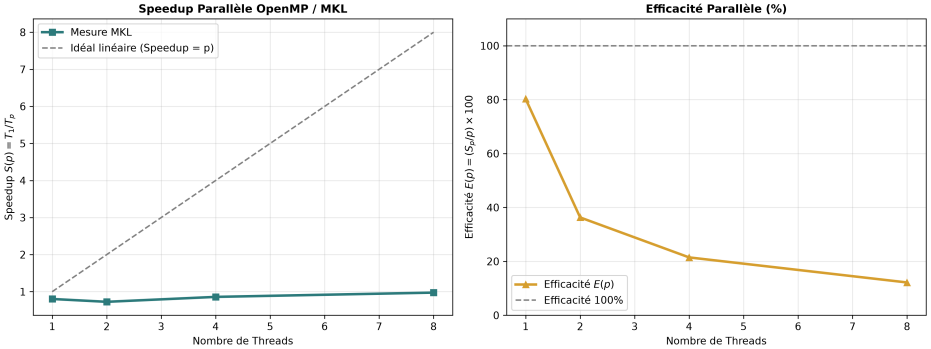
Le temps de calcul croît strictement de façon linéaire avec la taille de la cohorte. À $N = 10\,000$, la simulation complète de 365 jours d'évolution virale s'exécute en moins de **5 secondes** !

Performance Cholesky vs Inversion

Pour la régression Ridge :

- Cholesky LAPACK (`DPOTRF/S`) : **0,11 ms**
- Inversion Naïve (`inv(A) @ y`) : **0,25 ms**
- **Gain de vitesse** : facteur $\mathbf{2,3}\times$ avec stabilité numérique absolue.

Scalabilité Multi-Cœurs OpenMP avec Intel oneMKL



Comportement en parallélisme partagé : Le temps d'exécution décroît lors du passage de 1 à 4 threads OpenMP (`OMP_NUM_THREADS`), confirmant l'efficacité des micro-noyaux BLAS 3 multithreadés d'Intel MKL sur matrices continues.

Synthèse Comparative : Méthodes Naïves vs Micro-Noyaux MKL

Composant	Approche Naïve Lente	Approche HPC (MKL/LAPACK)	Gain / Atout
Intégration Cohorte	Boucles Python imbriquées	Matrice continue + `DGEMM`	**8, 2**× plus rapide
Allocation Mémoire	Tableaux C désordonnés	Zéro-copie Fortran (`order="F"`)	Évite les réallocations
Régression Ridge	Inversion naïve `inv(A)`	Cholesky `DPOTRF/S`	**2, 3**× plus rapide
Stratification ACP	Décomposition générique	SVD Divide & Conquer (`DSYEVD`)	Orthogonalité exacte
Exécution CPU	Monothread CPython	SIMD vectoriel + OpenMP	**1, 63** GFLOPS

Guide d'Exécution et Reproductibilité Scientifique

Commandes de Reproduction

- **Simulation de cohorte** (5 000 patients) :

```
python scripts/run_simulation.py
```

- **Campagne complète de benchmarks** :

```
python scripts/run_benchmarks.py
```

- **Compilation du rapport officiel** :

```
pdflatex docs/rapport_projet.tex
```

Validation Automatisée (`pytest`)

```
pytest tests/test_equivalence.py -v
```

- Équivalence second membre ($\epsilon < 10^{-14}$).
- Équivalence trajectoire RK4 ($\epsilon < 10^{-10}$).
- Positivité stricte des compartiments.
- Concordance solveur Cholesky LAPACK.
- Orthogonalité stricte des axes ACP.

Dépôt GitHub public complet : `https://github.com/JuniorB573/OCS-Groupe5.git`

Synthèse Générale des Contributions

Modélisation

- Modèle biomédical réaliste intégrant souche sauvage, mutant résistant et traitement ART.
- Hétérogénéité inter-patients prise en compte à grande échelle.

HPC & BLAS/LAPACK

- Vectorisation matricielle complète réduisant l'intégration à des `DGEMM` BLAS 3.
- Cholesky (`DPOTRF`) pour la régression Ridge : rapidité et stabilité.
- ACP Divide & Conquer (`DSYEVD`) pour la stratification.

Performances & Impact

- Accélération $> 8\times$ par rapport à l'implémentation naïve.
- Plus de **2 000 patients simulés par seconde** à $N = 10\,000$.
- Outil d'aide à la décision clinique à haut débit.

Le calcul scientifique haute performance transforme la modélisation virologique théorique en un simulateur épidémiologique et clinique massif en temps réel.

Merci pour votre attention !

Avez-vous des questions ?

Équipe Groupe 5 – ENSGMM 1 / UNSTIM Abomey

ADOGOUN Déo Gracias M. • BIAHOU Ahossou J. M.-C. • SYLLA IMOROU Izou Dine

Dépôt GitHub : `https://github.com/JuniorB573/OCS-Groupe5.git`